

DynaSLAM II: Tightly-Coupled Multi-Object Tracking and SLAM

Berta Bescos, Carlos Campos, Juan D. Tardós and José Neira

Abstract—The assumption of scene rigidity is common in visual SLAM algorithms. However, it limits their applicability in populated real-world environments. Furthermore, most scenarios including autonomous driving, multi-robot collaboration and augmented/virtual reality, require explicit motion information of the surroundings to help with decision making and scene understanding. We present in this paper DynaSLAM II, a visual SLAM system for stereo and RGB-D configurations that tightly integrates the multi-object tracking capability.

DynaSLAM II makes use of instance semantic segmentation and of ORB features to track dynamic objects. The structure of the static scene and of the dynamic objects is optimized jointly with the trajectories of both the camera and the moving agents within a novel bundle adjustment proposal. The 3D bounding boxes of the objects are also estimated and loosely optimized within a fixed temporal window. We demonstrate that tracking dynamic objects does not only provide rich clues for scene understanding but is also beneficial for camera tracking.

The project code will be released upon acceptance.

Index Terms—SLAM, semantics, tracking, dynamic objects

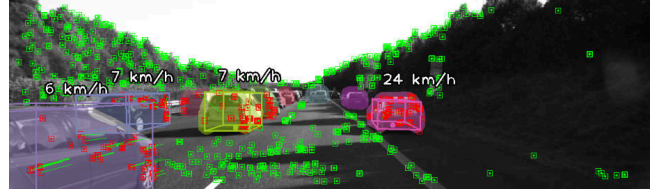
I. INTRODUCTION

Visual Simultaneous Localization and Mapping (SLAM) is the problem of creating a map of an unknown environment and estimating the robot pose within such map, only from the data streams of its on-board cameras. Most SLAM approaches assume a static scene and can only handle small fractions of dynamic content by labeling them as outliers to such static model [1]–[3]. Whilst the static premise holds for some robotic applications, it limits its use in populated situations for autonomous driving, service robots or AR/VR.

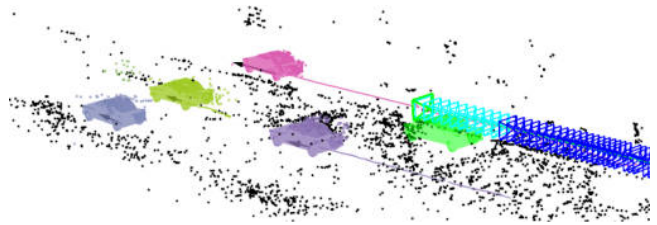
The problem of dealing with dynamic objects in SLAM has been widely targeted in recent years. The biggest part of the literature tackles this problem by detecting moving regions within the observed scene and rejecting such areas for the SLAM problem [4]–[7]. Some works process the image streams outside of the localization pipeline by translating the images that show dynamic content into realistic images with only static content [8], [9]. On the other hand, a small but growing part of the robotics community has addressed this issue by incorporating the dynamics of moving objects into the problem [10]–[13]. Whereas the two first groups mostly focus on achieving an accurate ego-motion estimation from the static scene, the goal of the latter group is twofold: they do not only solve the SLAM problem but also provide information about the poses of other dynamic agents.

This work has been supported by the Spanish Ministry of Economy and Competitiveness (projects PID2019-108398GB-I00, PGC2018-096367-B-I00 and FPI grant BES-2016-077836).

All authors are with the Instituto de Investigación en Ingeniería de Aragón (I3A), Universidad de Zaragoza, Zaragoza 50018, Spain {bbescos, campos, tardos, jneira}@unizar.es



(a) The 3D bounding box and the speed of the objects are inferred in the image. Static and dynamic key points are in green and red respectively.



(b) Joint estimation of the camera ego motion (green car), the sparse static 3D map (black points) and the trajectories of the dynamic objects. The cyan key frames allow to optimize the map dynamic structure, whereas the blue ones only optimize the camera pose and the static structure.

Fig. 1: Qualitative results with the KITTI tracking dataset.

Understanding surrounding dynamic objects is of crucial importance for the frontier requirements of emerging applications within AR/VR or autonomous systems navigation. Whereas it is tolerable to rule out minor movements in a quasi-static environment, most scenarios including autonomous driving, multi-robot collaboration and AR/VR require explicit motion information of the surroundings to aid in decision-making and scene understanding. For example, in virtual reality, dynamic objects need to be explicitly tracked to allow the interaction of virtual objects with moving instances in the real world. In autonomous driving scenarios, a car must not only localize itself but must also reliably perceive other vehicles and passers-by to avoid collisions.

The vast majority of the literature that specifically addresses this issue detect moving objects and track them separately from the SLAM formulation by using traditional multi-target tracking approaches [14]–[18]. Their accuracy highly depends on the camera pose estimation, which is more susceptible to failure in complex dynamic environments where the presence of reliable static structure is not guaranteed. In recent years, the robotics community has made its first steps towards addressing dynamic objects tracking jointly with visual SLAM, adding an extra layer of complexity to the problem. These systems are often tailored for special use cases and multiple priors are exploited to constraint the space solutions: road planar structure and planar object movement in driving scenarios, or even the use of object 3D models.

At this point, we take the opportunity to introduce DynaSLAM II. DynaSLAM II is an open-source stereo and RGB-D SLAM system for dynamic environments which simultaneously estimates the poses of the camera, the map and the trajectories of the scene moving objects. We propose a bundle adjustment solution that tightly optimizes the scene structure, the camera poses and the objects trajectories in a local-temporal window. The bounding boxes of the objects are also optimized in a decoupled formulation that allows to estimate the dimensions and the 6 DoF poses of the objects without being geared to any particular use case. We provide an exhaustive evaluation and comparison on the KITTI dataset [19], validating our proposal. An example of the output of our system DynaSLAM II can be seen in Fig. 1.

II. RELATED WORK

The traditional manner of addressing 3D multi-object tracking implies detecting and tracking the moving objects separately from the SLAM formulation [14]–[18]. Among them, Wang *et al.* [14] derived the Bayes formula of the SLAM with tracking of moving objects and provided a solid basis for understanding and solving this problem. Wangsiripitak *et al.* [15] proposed the parallel implementation of SLAM with a 3D object tracker: the SLAM provides the tracker with information to register map objects, and the tracker allows to mark features on objects. Rogers *et al.* [16] applied an EM technique to a graph based SLAM approach and allowed landmarks to be dynamic. More recently, Barsan *et al.* [17] presented a stereo-based dense mapping algorithm for urban environments that simultaneously reconstructs the static background and the moving objects. There is a new work by Rosinol *et al.* [18] that reconciles visual-inertial SLAM and dense mesh tracking, focusing mostly on humans, that shows impressive results in simulation. The main drawback of these approaches is that their accuracy is highly correlated with the estimation of the camera position. That is, if the camera pose estimation fails, which is quite likely in complex dynamic environments, multi-object tracking also fails directly.

The idea of simultaneously estimating camera motion and multiple moving objects motion originated from the SLAM-MOT work [20]. They established a mathematical framework to integrate a filtering-based SLAM and moving object tracking and demonstrated that it satisfied both navigation and safety requirements in autonomous driving. Later on, works that use RGB-D cameras followed up this idea to densely reconstruct static indoors scenes along with moving objects using pixel-wise instance segmentation, showing impressive results [21]–[23]. Since it is of crucial importance for dense approaches to obtain a highly accurate segmentation mask, the works Mask-Fusion [22] and MID-Fusion [23] refine it by assuming that human-made objects are largely convex.

Among the feature-based approaches, as is ours, few aim to merge information from static and dynamic objects into a single framework to boost estimation accuracy. Henein *et al.* [24] were among the first ones to tightly combine the problems of tracking dynamic objects and the camera ego motion. However, they only reported experiments on

synthetic data showing limited real results. Li *et al.* [10] use a CNN trained in an end-to-end manner to estimate the 3D pose and dimensions of cars, which is further refined together with camera poses. The use of data-driven approaches often provides excellent accuracy in 6 DoF object pose estimation, but also a loss of generality and thus, they can only track cars. Huge amounts of data would be required to track generic objects with their approach. The authors of CubeSLAM [11] showed impressive results with only a monocular camera by making use of a 3D bounding box proposal generation based on 2D bounding boxes and vanishing points. They assume that objects have a constant velocity within a hard-coded duration time interval and exploit object priors such as car sizes, road structure and planar non-holonomic object wheel motion models. Moreover, they only track objects whose 3D bounding box is observable, *i.e.*, only once two or more faces of the *cuboid-shape* object are seen. ClusterSLAM [25] proposes a SLAM back end with no scene priors to discover individual rigid bodies and compute their motions in dynamic environments. Since it acts as a back end instead of as a full system, its performance relies heavily on the landmark tracking and association quality. The same authors recently developed the full system ClusterVO [12], which models the object points with a probability of object belonging to deal with segmentation inaccuracies. Given that they assume no priors, they obtain good tracking results in indoor and outdoor scenes, but with an inaccurate estimation of the 3D bounding boxes. VDO-SLAM [13] is a recent work that uses dense optical flow to maximise the number of tracked points on moving objects. They implement a bundle adjustment with cameras, objects and points that gives good results but is computationally complex to run in real time.

In light of these advances, it is apparent that the feature-based SLAM community is searching for the best optimization formulation to combine cameras, objects and structure points. In our proposal, we use a tightly-coupled bundle adjustment formulation with new measurements between cameras, objects and points giving special attention to its computational complexity and number of parameters involved without introducing hard-coded priors. For this, we integrate instance semantic priors together with sparse image features. This formulation allows the estimation of both the camera, the map structure and the dynamic objects to be mutually beneficial at a low computational cost. On the other hand, part of the current literature focuses on the estimation of the point cloud structure of dynamic objects and of the trajectory of a random object reference [12], [13], [25], whereas another part of the literature seeks to find a common reference for objects of the same class as well as a more informative occupancy volume [10], [11]. We intend to carry out these two tasks independently in order to leverage the benefits of both and not suffer their disadvantages.

III. METHOD

DynaSLAM II builds on the popular ORB-SLAM2 [1]. It takes synchronized and calibrated stereo/RGB-D images as input, and outputs the camera and the dynamic objects

poses for each frame, as well as a spatial/temporal map containing the dynamic objects. For each incoming frame, pixel-wise semantic segmentation is computed and ORB features [26] are extracted and matched across stereo image pairs. We first associate the static and dynamic features with the ones from the previous frame and the map assuming a constant velocity motion for both the camera and the observed objects. Object instances are then matched based on the dynamic feature correspondences. The static matches are used to estimate the initial camera pose, and the dynamic ones yield the object’s SE(3) transform. Finally, the camera and objects trajectories, as well as the objects bounding boxes and 3D points are optimized over a sliding window with marginalization and a soft smooth motion prior. The different contributions and building blocks of DynaSLAM II are explained in the following subsections.

A. Notation

We will use the following notation: a stereo/RGB-D camera i has a pose $\mathbf{T}_{\text{CW}}^i \in \text{SE}(3)$ in the world coordinates $\mathbf{W}$ at time i (see Fig. 2). The camera i observes 1) static 3D map points $\mathbf{x}_w^l \in \mathbb{R}^3$ and 2) dynamic objects with pose $\mathbf{T}_{\text{WO}}^{k,i} \in \text{SE}(3)$ and linear and angular velocity $\mathbf{v}_i^k, \mathbf{w}_i^k \in \mathbb{R}^3$ at time i , all in object coordinates. Each observed object k contains dynamic objects points $\mathbf{x}_0^{j,k} \in \mathbb{R}^3$.

B. Objects Data Association

For each incoming frame, pixel-wise semantic segmentation is computed and ORB features [26] are extracted and matched across stereo image pairs. If an instance belongs to a dynamic class (vehicles, pedestrians and animals) and contains a high number of new nearby key points, an object is created. The key points are then assigned to the instance and the corresponding object. We first associate the static features with the ones from the previous frame and the map to initially estimate the camera pose. Next, dynamic features are associated with the dynamic points from the local map in two different ways: (a) if the velocity of the map objects is known, the matches are searched by reprojection assuming an inter-frame constant velocity motion, (b) if the objects velocity is not initialized or not enough matches are found following (a), we constraint the brute force matching to those features that belong to the most overlapping instance within consecutive frames. Note that our framework handles occlusions since current key points are matched to map objects instead of to the previous frame objects. A higher level association between instances and objects is also required. If most of the key points assigned to a new object are matched with points belonging to one map object, the two objects are attributed the same track id. Also, to render this high level association more robust, a parallel instance-to-instance matching is performed based on the IoU (Intersection over Union) of the CNN instances 2D bounding boxes.

The SE(3) pose of the first object of a track is initialized with the center of mass of the 3D points and with the identity rotation. To predict the poses of further objects from a track,

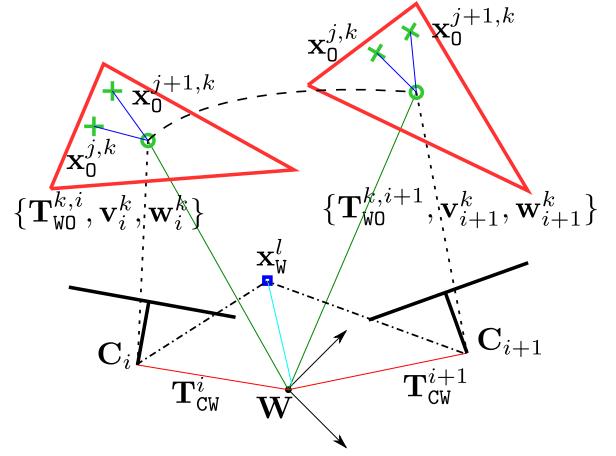


Fig. 2: Notation used to model the dynamic structure. The cameras i and $i+1$ observe the dynamic object k (---) and the static structure (----). The objects with poses $\mathbf{T}_{\text{WO}}^{k,i}$ and $\mathbf{T}_{\text{WO}}^{k,i+1}$ are the same moving body at consecutive observations (---).

we use a constant velocity motion model and refine the object pose estimate by minimizing the reprojection error.

The common formulation of the reprojection error in multi-view geometry problems for a camera i with pose $\mathbf{T}_{\text{CW}}^i \in \text{SE}(3)$ and a 3D map point l with homogeneous coordinates $\bar{\mathbf{x}}_w^l \in \mathbb{R}^4$ in the reference $\mathcal{W}$ with a stereo key point correspondence $\mathbf{u}_i^l = [u, v, u_R] \in \mathbb{R}^3$ is

$$\mathbf{e}_{\text{repr}}^{i,l} = \mathbf{u}_i^l - \pi_i(\mathbf{T}_{\text{CW}}^i \bar{\mathbf{x}}_w^l), \quad (1)$$

where π_i is the reprojection function for a rectified stereo/RGB-D camera that projects a 3D homogeneous point in the camera coordinates into the camera frame pixel. Unlike this formulation, which is valid for static representations, we propose to restate the reprojection error as

$$\mathbf{e}_{\text{repr}}^{i,j,k} = \mathbf{u}_i^j - \pi_i(\mathbf{T}_{\text{CW}}^i \mathbf{T}_{\text{WO}}^{k,i} \bar{\mathbf{x}}_0^{j,k}), \quad (2)$$

where $\mathbf{T}_{\text{WO}}^{k,i} \in \text{SE}(3)$ is the inverse pose of the object k in the world coordinates when the camera i is observing it, and $\bar{\mathbf{x}}_0^{j,k} \in \mathbb{R}^4$ represents the 3D homogeneous coordinates of the point j in its object reference k with observation in the camera i $\mathbf{u}_i^j \in \mathbb{R}^3$. This formulation enables us to optimize either jointly the poses of the cameras and of the different moving objects, as well as the positions of their 3D points.

C. Object-Centric Representation

Given the extra complexity and mainly the extra number of parameters that the task of tracking moving objects implies on top of the SLAM ones, it is of high importance to keep this number as reduced as possible to maintain a real-time performance. Modeling dynamic points as repeated 3D points by forming independent point clouds as in usual dynamic SLAM implementations results in a prohibitive amount of parameters. Given a set of N_c cameras, N_o dynamic objects with N_{op} 3D points each observed in all cameras, the number of parameters needed to track

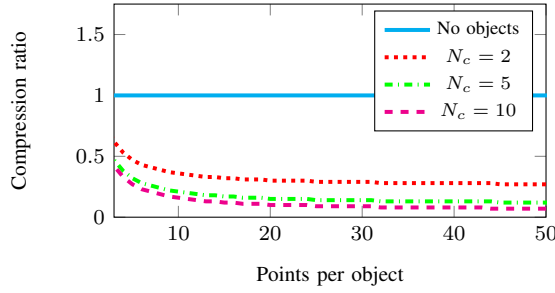


Fig. 3: Relationship between the number of parameters that is required when objects are used and when points belonging to objects are tracked independently (No objects).

dynamic objects becomes $N = 6N_c + N_c \times N_o \times 3N_{op}$ as opposed to $N = 6N_c + N_o \times 3N_{op}$ in conventional static SLAM representations. This number of parameters becomes prohibitive for long –and not so long– operations and deployment. If the concept of objects is introduced, 3D object points become unique and can be referred to their dynamic object. Therefore it is the pose of the object that is modelled along time and the number of required parameters shifts to $N' = 6N_c + N_c \times 6N_o + N_o \times 3N_{op}$. Fig. 3 shows the parameter compression ratio defined as $\frac{N'}{N}$ for 10 objects. This modelling of dynamic objects and points brings great savings in the number of utilized parameters.

D. Bundle Adjustment with Objects

Bundle Adjustment (BA) is known to provide accurate estimates of camera poses and sparse geometrical reconstruction, given a strong network of matches and good initial guesses. We hypothesize that BA might bring similar benefits if object poses are also jointly optimized (Fig. 4). The static map point 3D locations $\bar{\mathbf{x}}_w^l$ and camera poses $\mathbf{T}_{CW}^i$ are optimized by minimizing the reprojection error with respect to the matched key points $\mathbf{u}_i^l$ (Eqn. 1). Similarly, for dynamic representations, the object points $\bar{\mathbf{x}}_0^{j,k}$, the camera poses $\mathbf{T}_{CW}^i$ and the object poses $\mathbf{T}_{W0}^{k,i}$ can be refined by minimizing the reprojection error formulation presented in Eqn. 2.

In our implementation, a key frame can be inserted in the map for two different reasons: (a) the camera tracking is weak, (b) the tracking of any scene object is weak. The reasons for the former are the same ones than in ORB-SLAM2. The latter though happens if an object with a relatively large amount of features has few points tracked in the current frame. In this case, a key frame is inserted and creates a new object and new object points. If the camera tracking is not weak, this key frame would not introduce new static map points, and if the rest of dynamic objects have a stable tracking, new objects for these tracks would not be created. As for the optimization, if a key frame is inserted only because the camera tracking is weak, the local BA optimizes the currently processed key frame, all the key frames connected to it in the covisibility graph, and all the map points seen by those key frames, following the implementation of ORB-SLAM2. Regarding dynamic data, if a key frame is inserted only because the tracking of an

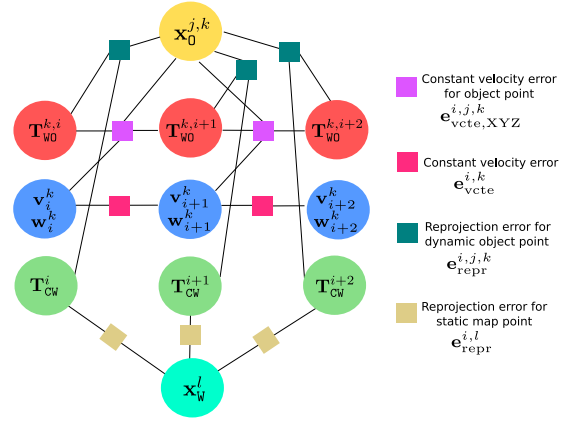


Fig. 4: BA factor graph representation with dynamic objects.

object is weak, the local BA optimizes the pose and velocity of this object and the camera along a temporal tail of 2 seconds together with its object points. Finally, if a key frame is inserted because the tracking of both the camera and objects are weak, camera poses, map structure, object poses, velocities and points are jointly optimized.

To avoid non-physically feasible object dynamics, a smooth trajectory is forced by assuming a constant velocity in consecutive observations. The linear and angular velocity of an object k at observation i are respectively denoted as $\mathbf{v}_i^k \in \mathbb{R}^3$ and $\mathbf{w}_i^k \in \mathbb{R}^3$. We define the following error term:

$$\mathbf{e}_{\text{vcte}}^{i,k} = \begin{pmatrix} \mathbf{v}_{i+1}^k - \mathbf{v}_i^k \\ \mathbf{w}_{i+1}^k - \mathbf{w}_i^k \end{pmatrix} \quad (3)$$

An additional error term is needed to couple the object velocities with the object poses and their corresponding 3D points. This term can be seen in Eqn. 4, where $\Delta \mathbf{T}_{0_k}^{i,i+1}$ is the pose transformation that the object k undergoes in the time interval $\Delta t_{i,i+1}$ between consecutive observations i and $i+1$.

$$\mathbf{e}_{\text{vcte,XYZ}}^{i,j,k} = \left(\mathbf{T}_{W0}^{k,i+1} - \mathbf{T}_{W0}^{k,i} \Delta \mathbf{T}_{0_k}^{i,i+1} \right) \bar{\mathbf{x}}_0^{j,k} \quad (4)$$

The term $\Delta \mathbf{T}_{0_k}^{i,i+1}$ is defined from the linear and angular velocity of the object k at time i ($\mathbf{v}_i^k$ and $\mathbf{w}_i^k$) as in Eqn. 5, where $\text{Exp} : \mathbb{R}^3 \rightarrow \text{SO}(3)$ is the exponential map for $\text{SO}(3)$.

$$\Delta \mathbf{T}_{0_k}^{i,i+1} = \begin{pmatrix} \text{Exp}(\mathbf{w}_i^k \Delta t_{i,i+1}) & \mathbf{v}_i^k \Delta t_{i,i+1} \\ \mathbf{0}_{1 \times 3} & 1 \end{pmatrix} \quad (5)$$

Finally, the following is our BA problem for a set of cameras in the optimizable local window $\mathcal{C}$ with each camera i observing a set of map points $\mathcal{MP}_i$ and an object set $\mathcal{O}_i$ containing each object k the set of object points $\mathcal{OP}_k$:

$$\min_{\theta} \sum_{i \in \mathcal{C}} \left(\sum_{l \in \mathcal{MP}_i} \rho(\|\mathbf{e}_{\text{repr}}^{i,l}\|_{\Sigma_l^i}^2) + \sum_{k \in \mathcal{O}_i} (\rho(\|\mathbf{e}_{\text{vcte}}^{i,k}\|_{\Sigma_{\Delta t}}^2) + \sum_{j \in \mathcal{OP}_k} (\rho(\|\mathbf{e}_{\text{repr}}^{i,j,k}\|_{\Sigma_j^i}^2) + \rho(\|\mathbf{e}_{\text{vcte,XYZ}}^{i,j,k}\|_{\Sigma_{\Delta t}}^2))) \right), \quad (6)$$

where ρ is the robust Huber cost function to downweigh outlier correspondences and Σ is the covariance matrix. In

the case of the reprojection error Σ is associated to the scale of the key point in the camera i observing the points l and j respectively. For the two other error terms Σ is associated to the time interval between two consecutive observations of an object, *i.e.*, the longer time the more uncertainty there is about the constant velocity assumption. The parameters to be optimized are $\theta = \{\mathbf{T}_{cw}^i, \mathbf{T}_{w0}^{k,i}, \mathbf{X}_w^l, \mathbf{X}_0^{j,k}, \mathbf{v}_i^k, \mathbf{w}_i^k\}$.

Fig. 5 shows the boolean Hessian matrix ($\mathbf{H}$) of the problem described. The Hessian can be built from the Jacobian matrices associated to each edge in the factor graph. In order to have a non-zero (i, j) block matrix, there must be an edge between i and j node in the factor graph. Notice the difference in the sparsity patterns of the map points and the object points. The size of the Hessian matrix is dominated by the number of map points N_{mp} and object points, which in typical problems is several orders of magnitude larger than the number of cameras and objects. Applying Schur complement trick and solving the system has a run-time complexity of $\mathcal{O}(N_c^3 + N_c^2 N_{mp} + N_c N_o N_{op})$, where either the second or third term will dominate the cost depending on the number of static and dynamic points.

E. Bounding Boxes

Some approaches in the current literature would consider the tracking of dynamic objects to be complete with the contributions so far. That is, for every dynamic object in the scene we have an estimation of the trajectory of the centroid of its map points when it was first observed, as well as a point cloud representation. Examples of these works are ClusterSLAM [25] and VDO-SLAM [13]. However, we believe that it is also of key importance to find a common spatial reference for objects of the same semantic class, as well as an estimate of their dimensions and space occupancy.

Alternatively, the basis of CubeSLAM [11] and of the work by Li *et al.* [10] is the discovery of object bounding boxes. Only once bounding boxes are discovered, are objects tracked along frames. That is, if the camera viewing angle does not allow to estimate an object bounding box (partial view), the object tracking does not take place. Whereas this is not a problem for Li *et al.* because CNNs are by nature robust to partial views of objects, CubeSLAM struggles to initialize bounding boxes from views of occluded objects.

We propose to decouple the estimation of the trajectories and the bounding boxes of the dynamic objects. The former provides the system tracking with rich clues for ego-motion estimation, and the conjunction of both are useful to understand the dynamics of the surroundings. The output of the data association and the BA stages contains the camera poses, the structure of the static scene and the dynamic objects, and the 6 DoF trajectory of one point for each object. This one point is the center of mass of the object 3D points when it is first observed. Even though the center of mass changes along time with new points observations, the object pose that is tracked and optimized is referred to this first center of mass. To have a full understanding of the moving surroundings, it is of high importance to know the objects dimensions and space occupancy. Tackling the two problems independently

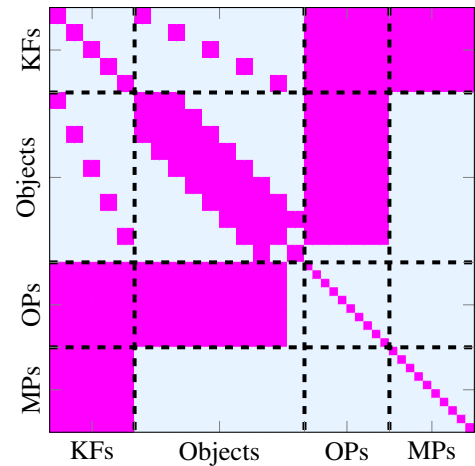


Fig. 5: Hessian matrix for 5 key frames (KFs), 1 object with 10 object points (OPs) and 10 static map points (MPs).

allows to track dynamic objects from the first frame in which they appear independently of the camera-object view point.

We initialize an object bounding box by searching two perpendicular planes that fit roughly the majority of the object points. We hypothesize that even though objects are not always perfect cuboids, many can approximately fit a 3D bounding box. In the case in which only one plane is found, we add a prior on the rough dimensions of the non-observable direction that is related to the object class. This procedure is done within a RANSAC scheme: we choose the computed 3D bounding box that has the largest IoU of its image projection with the CNN 2D bounding box. This bounding box is computed once for every object track.

To refine the bounding box dimensions and its pose relative to the object tracking reference, an image-based optimization is performed within a temporal window. This optimization seeks to minimize the distance between the 3D bounding box image projection and the CNN 2D bounding box prediction. Given that this problem is not observable for less than three views of an object, this is only performed once an object has at least three observing key frames. Also, to constraint the solution space in case the view of an object makes this problem non-observable (*e.g.*, a car observed from the back), a soft prior about the object dimensions is included. Since this prior is tightly related to the object class, we believe that adding this soft prior does not mean a loss of generality. Finally, the initial bounding box pose is set as a prior so that the optimization solution remains close.

IV. EXPERIMENTS

In this section we detail the experiments carried out to test DynaSLAM II. It is divided in two main blocks: one that assesses the effect of tracking objects on the estimation of camera motion (Subsection IV-A), and one that analyzes the multi-object tracking performance (Subsection IV-B).

A. Visual Odometry

For the visual odometry experiments we have chosen the KITTI tracking (Table I) and raw (Table II) datasets [19].

seq	ORB-SLAM2			DynaSLAM			VDO-SLAM			Ours		
	ATE [m]	RPE _t [m/f]	RPE _R [°/f]	ATE [m]	RPE _t [m/f]	RPE _R [°/f]	ATE [m]	RPE _t [m/f]	RPE _R [°/f]	ATE [m]	RPE _t [m/f]	RPE _R [°/f]
0000	1.32	0.04	0.06	1.35	0.04	0.06	-	0.05	0.05	1.29	0.04	0.06
0001	1.95	0.05	0.04	2.42	0.05	0.04	-	0.12	0.04	2.31	0.05	0.04
0002	0.95	0.04	0.03	1.04	0.04	0.03	-	0.04	0.02	0.91	0.04	0.02
0003	0.74	0.07	0.04	0.78	0.07	0.04	-	0.09	0.04	0.69	0.06	0.04
0004	1.44	0.07	0.06	1.52	0.07	0.06	-	0.11	0.05	1.42	0.07	0.06
0005	1.23	0.06	0.03	1.22	0.06	0.03	-	0.10	0.02	1.34	0.06	0.03
0006	0.19	0.02	0.04	0.19	0.02	0.04	-	0.02	0.05	0.19	0.02	0.04
0007	2.47	0.05	0.07	2.69	0.05	0.07	-	-	-	3.10	0.05	0.07
0008	1.40	0.08	0.04	1.29	0.08	0.04	-	-	-	1.68	0.10	0.04
0009	4.00	0.06	0.05	3.55	0.06	0.05	-	-	-	5.02	0.06	0.06
0010	1.68	0.07	0.04	1.84	0.07	0.04	-	-	-	1.30	0.07	0.03
0011	0.97	0.04	0.03	1.05	0.04	0.03	-	-	-	1.03	0.04	0.03
0013	1.18	0.04	0.05	1.18	0.04	0.05	-	-	-	1.10	0.04	0.04
0014	0.13	0.03	0.08	0.13	0.03	0.08	-	-	-	0.12	0.03	0.08
0018	0.89	0.05	0.03	1.00	0.05	0.03	-	0.07	0.02	1.09	0.05	0.02
0019	2.31	0.05	0.03	2.35	0.05	0.03	-	-	-	2.25	0.05	0.03
0020	16.80	0.11	0.07	1.10	0.05	0.04	-	0.16	0.03	1.36	0.07	0.04
mean	2.33	0.055	0.046	1.45	0.051	0.045	-	0.084	0.036	1.54	0.053	0.043

TABLE I: Egomotion comparison on the KITTI tracking dataset. Results of sequences without egomotion are not shown.

They contain gray-scale and RGB stereo sequences of urban and road scenes recorded from a car perspective with circulating vehicles and pedestrians, as well as its GPS data.

Both tables detail comparisons of our system’s performance against ORB-SLAM2 and our previous work DynaSLAM [4]. ORB-SLAM2 is the base SLAM system on which we build DynaSLAM II, and does not specifically address dynamic objects. DynaSLAM adds ORB-SLAM2 the capability to detect the features belonging to dynamic objects and classes but uniquely ignores them and does not track them. The difference in the results of ORB-SLAM2 and DynaSLAM gives an idea of how dynamic each sequence is. Theoretically, if dynamic objects are representative in the scene and they are in circulation, DynaSLAM has better performance, as can be seen in sequences 0020 and 1003-0047 in Tables I and II respectively. However, if dynamic objects are representative in the scene but not in motion, *e.g.*, parked cars, DynaSLAM shows a larger trajectory error. This happens because the features belonging to the static vehicles, which are useful for pose estimation and usually lay in nearby scene regions, are not utilized. This can be seen for example in the sequence 0001 in Table I. Besides that, DynaSLAM II achieves a performance better than both ORB-SLAM and DynaSLAM in these two types of scenarios in many of the evaluated sequences. On the one hand, when dynamic instances are moving, DynaSLAM II successfully estimates the velocity of the corresponding objects and provide the BA with rich clues for camera pose estimation when the static representation is not sufficient. This often occurs when dynamic objects occlude nearby scene regions and thus static features only provide valuable hints for accurately estimating the camera rotation. On the other hand, when dynamic classes instances are static, DynaSLAM II tracks their features estimating that their velocity is close to zero. Consequently, these object points act much like static points.

Tables I and II present our ego motion results compared to those of state-of-the-art systems that also track dynamic

objects in a joint SLAM framework. ClusterSLAM [25] acts as a back end rather than a SLAM system and is highly dependent on the camera poses initial estimates. ClusterVO [12] and VDO-SLAM [13] are SLAM systems as ours with the multi-object tracking capability. The former can handle stereo and RGB-D data, whereas the latter only handles RGB-D. The reported errors are given with different metrics so that we can directly use the values that the authors provide. DynaSLAM II achieves in all sequences a lower translational relative error (RPE_t) than that of VDO-SLAM. However, VDO-SLAM usually achieves a lower rotational pose error. Since far points are the ones that provide the richest clues for rotation estimation, we believe that this difference in accuracy does not depend on the object tracking performance and is therefore due to the underlying camera pose estimation algorithm and sensor suite. Regarding the performance of ClusterVO, it achieves an accuracy which is in most sequences quite similar to ours.

B. Multi-Object Tracking

Once the utility of tracking dynamic objects for ego motion estimation is demonstrated, we have chosen once again the KITTI tracking dataset [19] to validate our multi-object tracking results. The trajectories and the 3D bounding boxes of the dynamic objects are provided thanks to expensive manual annotations on LIDAR 3D point clouds.

First of all, we would like to draw the attention of the reader to Fig. 1 to have a look at our qualitative results on this dataset. The bounding boxes of the two purple cars on the left are well estimated despite their partial view. This scene is also challenging because the other two front cars are far from the camera and yet are correctly tracked.

In the last decade Bernardin *et al.* [27] introduced the CLEAR MOT metrics to allow for objective comparison of tracker characteristics, focusing on their precision in estimating object locations, their accuracy in recognizing object configurations and their ability to consistently label

seq	ORB-SLAM2			DynaSLAM			ClusterSLAM			ClusterVO			Ours		
	ATE [m]	RPE _t [m]	RPE _R [rd]	ATE [m]	RPE _t [m]	RPE _R [rd]	ATE [m]	RPE _t [m]	RPE _R [rd]	ATE [m]	RPE _t [m]	RPE _R [rd]	ATE [m]	RPE _t [m]	RPE _R [rd]
0926-0009	0.83	1.85	0.01	0.81	1.80	0.01	0.92	2.34	0.03	0.79	2.98	0.03	0.85	1.87	0.01
0926-0013	0.32	1.04	0.01	0.30	0.99	0.01	2.12	5.50	0.07	0.26	1.16	0.01	0.29	0.93	0.00
0926-0014	0.50	1.22	0.01	0.60	1.62	0.01	0.81	2.24	0.03	0.48	1.04	0.01	0.48	1.35	0.01
0926-0051	0.38	1.16	0.00	0.46	1.17	0.00	1.19	1.44	0.03	0.81	2.74	0.02	0.44	1.14	0.00
0926-0101	2.97	13.63	0.03	3.52	15.14	0.03	4.02	12.43	0.02	3.18	12.78	0.02	4.33	15.02	0.04
0929-0004	0.62	1.38	0.01	0.56	1.36	0.01	1.12	2.78	0.02	0.40	1.77	0.02	0.64	1.41	0.01
1003-0047	20.49	32.59	0.08	2.87	5.95	0.02	10.21	8.94	0.06	4.79	6.54	0.05	3.03	6.85	0.02
mean	3.73	7.55	0.02	1.30	4.00	0.01	2.91	5.10	0.04	1.53	4.14	0.02	1.44	4.08	0.01

TABLE II: Egomotion comparison on the KITTI raw dataset

objects over time. Whereas these metrics are well established in the computer vision and robotics communities and provide valuable insights about the per-frame performance of trackers, they do not take into account the quality of the tracked object trajectories. We suggest that to correctly evaluate multi-object tracking within a SLAM framework, one needs to report the CLEAR MOT metric MOTP¹ as well as the common trajectory error metrics. Most related works on SLAM and multi-object tracking only report the CLEAR MOT metric MOTP [10]–[12] and besides that, the authors of VDO-SLAM [13] uniquely report the relative pose error of all objects trajectories of one sequence as a single ensemble. We think that, to facilitate comparison, this metric should be instead reported for individual trajectories.

Table III shows an evaluation of all object detections in the KITTI tracking dataset with the KITTI 3D object detection benchmark. This allows us to directly compare our multi-object tracking results to those of other state-of-the-art similar systems (Table III). The CNNs of Chen *et al.* [28] and specially of Li *et al.* [10] achieve excellent results thanks to the single-view network accuracy itself and the multi-view refinement approach of the latter one, to the detriment of a generality loss. On the other hand, the accuracy of Barsan *et al.* [17] and of Huang *et al.* [12] in detecting bounding boxes is remarkable, but is very sensitive to object truncation and occlusion. Our results show that we can handle objects truncation and occlusion with a minor loss in precision. However, less bounding boxes are usually discovered. Our intuition is that our system feature-based nature renders this step specially challenging, opposite to the work by Barsan *et al.* [17], which first computes dense stereo matching.

	MOTP _{BV}			MOTP _{3D}		
	Easy	Moderate	Hard	Easy	Moderate	Hard
[28]	81.34 %	70.70 %	66.32 %	80.62 %	70.01 %	65.76 %
[10]	88.07 %	77.83 %	72.73 %	86.57 %	74.13 %	68.96 %
[17]	71.83 %	47.16 %	40.30 %	64.51 %	43.70 %	37.66 %
[12]	74.65 %	49.65 %	45.62 %	55.85 %	38.93 %	33.55 %
Ours	64.69 %	58.75 %	58.36 %	53.14 %	48.66 %	48.57 %

TABLE III: MOTP evaluation on the KITTI tracking dataset. The categories Easy, Moderate and Hard are based on the 2D bounding boxes height, occlusion and truncation level.

¹MOTP stands for multiple object tracking precision. It is the predictions precision computed with any given cost function over the number of TPs.

To evaluate our estimation of object trajectories, in Table IV we have chosen the 12 longest sequences of the KITTI tracking dataset whose 2D detections are neither occluded nor truncated, and whose height is at least 40 pixels. These chosen objects are labeled with their ground-truth object id. For each of these ground truth trajectories we look for the most overlapping bounding boxes in our estimations (the overlapping has to be of at least 25 %). In the case of the trajectory metrics (ATE and RPE) and the 2D MOTP, this overlapping is computed as the IoU of the 3D bounding boxes projected over the current frame. For the other two evaluations (BV and 3D), the overlapping is computed as the IoU of the bounding boxes in bird view and in 3D respectively. This evaluation gives an idea of our framework tracking performance and our bounding boxes quality. Regarding the true positives percentage, we can see that objects are tracked for the majority of their trajectory. Missing detections occur because the objects lay far from the camera and the stereo matching does not provide enough features for a rich tracking. It is important to notice that the accuracy of the passersby tracking is lower than that of the cars due to their non-rigid shape (sequence 0017). The trajectory errors of the cars are acceptable but they are far from the ego-motion estimation performance. Our intuition is that our algorithm feature-based nature renders the bounding box estimation specially challenging. A larger amount of 3D points would always provide richer clues for object tracking.

C. Timing Analysis

To complete the evaluation of our proposal, Table V shows the average computational time for its different building blocks. The timing of DynaSLAM II is highly dependent on the number of objects to be tracked. In sequences like KITTI tracking 0003 there are only two objects at a time as maximum and it can thus run at 12 fps. However, the sequence 0020 can have up to 20 objects at a time and its performance is seen slightly compromised, but still achieves a real time performance at ~ 10 fps. We do not include within these numbers the computational time of the semantic segmentation CNN since it depends on the GPU power and CNN model complexity. Algorithms such as YOLACT [29] can run in real time and provide high-quality instance masks.

Finally, the last rows of Table V collect the average timing results for systems that jointly perform SLAM and multi-object tracking in the KITTI dataset. DynaSLAM II is the only system that can provide at present a real-time solution.

sequence object id (class)	0003 1 (car)	0005 31 (car)	0010 0 (car)	0011 0 (car) 35 (car)		0018 2 (car) 3 (car)		0019 63 (car) 72 (car)		0020 0 (car)	0020 12 (car)	122 (car)
ATE [m]	0.69	0.51	0.95	1.05	1.25	1.10	1.13	0.86	0.99	0.56	1.18	0.87
RPE _t [m/m]	0.34	0.26	0.40	0.43	0.89	0.30	0.55	1.45	1.12	0.45	0.40	0.72
RPE _r [°/m]	1.84	13.50	2.84	12.51	16.64	9.27	20.05	48.80	3.36	1.30	6.19	5.75
2D	TP (%)	50.00	28.96	81.63	72.65	53.17	86.36	53.33	35.26	29.11	63.68	34.90
	MOTP [%]	71.79	60.30	73.51	74.78	65.25	74.81	70.94	63.50	62.59	78.54	78.76
BV	TP (%)	39.34	14.48	70.41	61.66	19.05	67.05	21.75	29.48	29.43	43.78	34.51
	MOTP [%]	56.61	46.84	47.60	50.74	31.95	45.47	41.45	45.69	55.48	45.00	48.05
3D	TP (%)	38.53	11.45	68.37	52.28	6.35	62.12	16.84	26.48	29.43	31.84	29.02
	MOTP [%]	48.20	34.20	40.28	47.35	26.02	34.80	35.80	33.89	39.81	46.15	44.43

TABLE IV: Objects motion comparison on the KITTI tracking dataset.

Sequence	Building block	Time [ms]
KITTI tracking 0003	Tracking thread	80.10 ± 0.78
	Local BA	61.37 ± 6.70
	Bounding Boxes BA	0.07 ± 0.01
KITTI tracking 0020	Tracking thread	94.56 ± 1.27
	Local BA	65.03 ± 17.72
	Bounding Boxes BA	0.60 ± 0.05
	[10] [13] [25] [12] Ours	
fps	5.8 5 - 8 7 8 10 - 12	

TABLE V: DynaSLAM II average computational time.

V. CONCLUSIONS AND FUTURE WORK

We have proposed an object level SLAM system with novel measurement functions between cameras, objects and 3D map points. This allows us to track dynamic objects and tightly optimize the trajectories of self and surroundings to let both estimations be mutually beneficial. We decouple the problem of object tracking from that of bounding boxes estimation and, differently from other works, we do not make any assumptions about the objects motion, pose or model. Our experiments show that DynaSLAM II achieves a state-of-the-art accuracy at real time performance, which renders our framework suitable for a large number of applications.

The feature-based core of our system limits its ability to discover accurate 3D bounding boxes, and also to track objects with low texture. Fully exploiting the dense visual information would certainly push these limits forward. We would also like to explore the –even more– challenging task of multi-object tracking and SLAM with only a monocular camera. This is an interesting direction since dynamic object tracking can provide rich clues about the scale of the map.

REFERENCES

- [1] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras,” *IEEE T-RO*, 2017.
- [2] J. Engel, V. Koltun, and D. Cremers, “Direct sparse odometry,” *Transactions on pattern analysis and machine intelligence*, 2017.
- [3] C. Forster, M. Pizzoli, and D. Scaramuzza, “SVO: Fast semi-direct monocular visual odometry,” in *IEEE ICRA*, pp. 15–22, 2014.
- [4] B. Bescos, J. M. Fàcil, J. Civera, and J. Neira, “DynaSLAM: Tracking, mapping, and inpainting in dynamic scenes,” *IEEE RA-L*, 2018.
- [5] Y. Sun, M. Liu, and M. Q.-H. Meng, “Improving RGB-D SLAM in dynamic environments: A motion removal approach,” *RAS*, 2017.
- [6] S. Li and D. Lee, “RGB-D SLAM in dynamic environments using static point weighting,” *IEEE Robotics and Automation Letters*, 2017.
- [7] L. Xiao, J. Wang, X. Qiu, Z. Rong, and X. Zou, “Dynamic-SLAM: Semantic monocular visual localization and mapping based on deep learning in dynamic environment,” *RAS*, vol. 117, 2019.
- [8] B. Bescos, J. Neira, R. Siegwart, and C. Cadena, “Empty cities: Image inpainting for a dynamic-object-invariant space,” in *IEEE ICRA*, 2019.
- [9] B. Bešić and A. Valada, “Dynamic Object Removal and Spatio-Temporal RGB-D Inpainting via Geometry-Aware Adversarial Learning,” *arXiv:2008.05058*, 2020.
- [10] P. Li, T. Qin, *et al.*, “Stereo vision-based semantic 3D object and ego-motion tracking for autonomous driving,” in *IEEE ECCV*, 2018.
- [11] S. Yang and S. Scherer, “CubeSLAM: Monocular 3-D object SLAM,” *IEEE Transactions on Robotics*, vol. 35, no. 4, pp. 925–938, 2019.
- [12] J. Huang, S. Yang, T.-J. Mu, and S.-M. Hu, “ClusterVO: Clustering Moving Instances and Estimating Visual Odometry for Self and Surroundings,” in *IEEE CVPR*, pp. 2168–2177, 2020.
- [13] J. Zhang, M. Henein, R. Mahony, and V. Ila, “VDO-SLAM: A Visual Dynamic Object-aware SLAM System,” *arXiv:2005.11052*, 2020.
- [14] C.-C. Wang, C. Thorpe, and S. Thrun, “Online simultaneous localization and mapping with detection and tracking of moving objects: Theory and results from a ground vehicle in crowded urban areas,” in *IEEE International Conference on Robotics and Automation*, 2003.
- [15] S. Wangsiripitak and D. W. Murray, “Avoiding moving outliers in visual SLAM by tracking moving objects,” in *ICRA*, IEEE, 2009.
- [16] J. G. Rogers, A. J. Trevor, C. Nieto-Granda, and H. I. Christensen, “SLAM with expectation maximization for moveable object tracking,” in *IEEE International Conf. on Intelligent Robots and Systems*, 2010.
- [17] I. A. Bărsan, P. Liu, M. Pollefeys, and A. Geiger, “Robust dense mapping for large-scale dynamic environments,” in *IEEE ICRA*, 2018.
- [18] A. Rosinol, A. Gupta, M. Abate, J. Shi, and L. Carlone, “3D Dynamic Scene Graphs: Actionable Spatial Perception with Places, Objects, and Humans,” *arXiv:2002.06289*, 2020.
- [19] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, “Vision meets robotics: The kitti dataset,” *IJRR*, vol. 32, no. 11, pp. 1231–1237, 2013.
- [20] C.-C. Wang, C. Thorpe, S. Thrun, M. Hebert, and H. Durrant-Whyte, “Simultaneous localization, mapping and moving object tracking,” *The International Journal of Robotics Research*, vol. 26, no. 9, 2007.
- [21] M. Rünz and L. Agapito, “Co-fusion: Real-time segmentation, tracking and fusion of multiple objects,” in *IEEE ICRA*, pp. 4471–4478, 2017.
- [22] M. Runz, M. Buffier, and L. Agapito, “Maskfusion: Real-time recognition, tracking and reconstruction of multiple moving objects,” in *IEEE International Symposium on Mixed and Augmented Reality*, 2018.
- [23] B. Xu, W. Li, D. Tzoumanikas, M. Bloesch, A. Davison, and S. Leutenegger, “MID-fusion: Octree-based object-level multi-instance dynamic SLAM,” in *IEEE ICRA*, pp. 5231–5237, 2019.
- [24] M. Henein, G. Kennedy, R. Mahony, and V. Ila, “Exploiting rigid body motion for SLAM in dynamic environments,” *IEEE ICRA*, 2018.
- [25] J. Huang, S. Yang, Z. Zhao, Y.-K. Lai, and S.-M. Hu, “ClusterSLAM: A SLAM Backend for Simultaneous Rigid Body Clustering and Motion Estimation,” in *IEEE ICCV*, pp. 5875–5884, 2019.
- [26] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, “ORB: An efficient alternative to SIFT or SURF,” in *ICCV*, IEEE, 2011.
- [27] K. Bernardin and R. Stiefel, “Evaluating multiple object tracking performance: the CLEAR MOT metrics,” *EURASIP JIVP*, 2008.
- [28] X. Chen, K. Kundu, Y. Zhu, H. Ma, S. Fidler, and R. Urtasun, “3D object proposals using stereo imagery for accurate object class detection,” *IEEE TPAMI*, vol. 40, no. 5, pp. 1259–1272, 2017.
- [29] D. Bolya, C. Zhou, F. Xiao, and Y. J. Lee, “Yolact: Real-time instance segmentation,” in *ICCV*, pp. 9157–9166, IEEE, 2019.